TRAVAUX DIRIGÉS — INFORMATIQUE

Géolocalisation, calcul de distances et triangulation en Python

Eléments de corrigé

Ce TD s'inscrit dans une problématique de géolocalisation appliquée : déterminer la position d'un point inconnu à partir de distances mesurées depuis plusieurs points de référence (triangulation). Vous travaillerez avec des données réelles de villes françaises et des scripts Python utilisant les bibliothèques folium et pandas.

Objectifs pédagogiques

- Lire et exploiter un fichier CSV avec Python (modules csv et pandas)
- Générer des cartes interactives avec la bibliothèque folium
- Appliquer le principe de triangulation géographique
- Analyser, corriger et améliorer du code Python existant

Fichiers fournis

Fichier	Description
`villes_coo.csv`	Base de données de villes françaises avec coordonnées GPS
`villes_vol_oiseau01.py`	Script v1 : calcul de distance et tracé du segment sur la carte
`villes_vol_oiseau02.py`	Script v2 : ajout du marqueur de distance sur la carte
`triangulation.py`	Script de triangulation à 3 villes avec saisie utilisateur
`triangulation_autre_approche.py`	Refactorisation du script de triangulation en fonctions

PARTIE 1 — DISTANCE À VOL D'OISEAU

Rappels théoriques — Formule de Haversine

La formule de Haversine calcule la distance en km entre deux points GPS sur la sphère terrestre (R ≈ 6371 km). Soient P1(lat1, lon1) et P2(lat2, lon2) (coordonnées en radians) :

```
dlat = lat2 - lat1
dlon = lon2 - lon1
a = sin(dlat/2)² + cos(lat1) × cos(lat2) × sin(dlon/2)²
c = 2 × atan2(sqrt(a), sqrt(1-a))
distance = R × c
```

La conversion degrés → radians s'effectue avec math.radians() en Python.

Exercice 1.1 — Analyse des scripts fournis

Q1. Ouvrez le fichier **villes_coo.csv**. Quel est le séparateur utilisé ? Quelle est la structure des données (colonnes) ?

Séparateur : point-virgule (;).

Le fichier contient 3 colonnes :

- ville : nom de la ville (chaîne de caractères)
- latitude : coordonnée GPS nord-sud (nombre décimal)
- longitude : coordonnée GPS est-ouest (nombre décimal)

Exemple : Angers;47.476657084;-0.556221025

Q2. Comparez **villes_vol_oiseau01.py** et **villes_vol_oiseau02.py**. Quelle est l'unique différence fonctionnelle ? Pourquoi est-elle utile pour l'utilisateur de la carte ?

Dans la v2, un marqueur est ajouté au milieu du segment avec la distance comme popup :

```
folium.Marker([(lat1+lat2)/2, (lon1+lon2)/2],
popup=texte_distance).add_to(carte)
```

Dans la v1, la distance n'apparaissait que dans le terminal. La v2 l'affiche directement sur la carte à un clic, rendant le résultat visible sans quitter l'interface cartographique.

Q3. Dans **calculer_distance(),** pourquoi est-il nécessaire de convertir les coordonnées en radians ? Que se passerait-il sans cette conversion ?

Les fonctions trigonométriques Python (math.sin, math.cos, math.atan2) attendent des radians. Sans conversion, math.sin(48) interpréterait 48 comme 48 radians au lieu de 48°, produisant des valeurs trigonométriques incorrectes. Les distances calculées seraient totalement erronées (plusieurs ordres de grandeur).

Exercice 1.2 — Modification et extension du code

Q4. Réécrivez la fonction obtenir_coordonnees_de_csv() de **villes_vol_oiseau01.py** en utilisant le module csv standard (sans pandas). La fonction doit retourner None si la ville est introuvable.

```
import csv

def obtenir_coordonnees_de_csv(nom_ville, fichier_csv):
    with open(fichier_csv, 'r', encoding='utf-8-sig') as f:
        reader = csv.DictReader(f, delimiter=';')
        for row in reader:
            if row['ville'] == nom_ville:
                return float(row['latitude']), float(row['longitude'])
    print(f"Ville '{nom_ville}' introuvable.")
    return None
```

Q5. Ajoutez une gestion d'erreur dans main() de **villes_vol_oiseau02.py** : si l'une des villes n'est pas trouvée, afficher un message précisant laquelle est manquante, sans faire planter le programme.

```
coord1 = obtenir_coordonnees_de_csv(ville1, fichier_csv)
coord2 = obtenir_coordonnees_de_csv(ville2, fichier_csv)

if coord1 is None:
    print(f"Erreur : '{ville1}' non trouvée dans {fichier_csv}.")
elif coord2 is None:
    print(f"Erreur : '{ville2}' non trouvée dans {fichier_csv}.")
else:
    # ... suite du traitement normal
```

PARTIE 2 — TRIANGULATION GÉOGRAPHIQUE

Rappels théoriques — Principe de la triangulation

La triangulation localise un point inconnu X à partir de distances connues depuis 3 points de référence A, B, C. X se trouve à l'intersection des 3 cercles de centres A, B, C et de rayons d(X,A), d(X,B), d(X,C).

Deux cercles se coupent en 2 points (ambiguïté). Un 3e cercle lève cette ambiguïté. En pratique, les imprécisions font que les cercles délimitent une zone d'incertitude plutôt qu'un point exact.

Exercice 2.1 — Analyse du script triangulation.py

Q6. Quel est le rôle de **obtenir_coordonnees_de_csv()** dans **triangulation.py** ? En quoi diffère-t-elle de celle des scripts villes_vol_oiseau ?

Dans triangulation.py, la fonction utilise le module csv standard (lecture ligne par ligne, séparateur ';') et retourne le tuple (None, None) en cas d'échec. Les scripts villes_vol_oiseau utilisent pandas.read_csv(), plus concis mais avec une dépendance externe supplémentaire. La version csv est plus légère et portable.

Q7. Dans **triangulation.py**, le rayon saisi en km est multiplié par 1000. Expliquez pourquoi. Quelle est l'unité du paramètre radius dans folium ?

folium.Circle() (via Leaflet.js) exprime le radius en mètres. L'utilisateur saisit en km, donc la conversion km × 1000 = m est indispensable. Sans elle, un rayon de 200 km s'afficherait comme 200 mètres — un cercle quasiment invisible sur la carte.

Q8. Pourquoi 3 cercles sont-ils nécessaires pour la triangulation ? Dans quels cas 2 cercles ne donnent-ils pas une solution unique ?

Deux cercles se coupent en général en 2 points symétriques : on ne peut pas déterminer lequel est le bon. Un 3e cercle lève l'ambiguïté car seul l'un des 2 points sera sur ce cercle. Cas particuliers pour 2 cercles : s'ils sont concentriques (0 ou infini de solutions) ou disjoints/l'un dans l'autre (0 solution), il n'y a aucun point d'intersection.

Exercice 2.2 — Amélioration du code de triangulation

Q9. Le fichier **triangulation_autre_approche.py** propose une refactorisation. Identifiez les 2 fonctions ajoutées et expliquez l'intérêt de cette décomposition.

Les 2 fonctions ajoutées sont :

- **add_circles_and_markers(map, villes, rayons)** : encapsule l'ajout des cercles et marqueurs, évitant la répétition pour chaque ville.
- **save_map(map, filename)** : isole la sauvegarde de la carte.

Intérêt : principe de responsabilité unique (SRP), meilleure lisibilité, réutilisabilité et testabilité de chaque composant de manière indépendante.

Q10. **triangulation_autre_approche.py** contient un bug : dans add_circles_and_markers(), la variable coords est utilisée sans être passée en paramètre. Proposez une correction complète.

```
def add_circles_and_markers(map, villes, rayons, coords):
    colors = ['blue', 'red', 'green']
    for ville, rayon, coord, color in zip(villes, rayons, coords, colors):
        folium.Circle(location=coord, radius=rayon*1000,
                      color=color, fill=True, fill_opacity=0.2).add_to(map)
        folium.Marker(location=coord, popup=ville).add_to(map)
```

Q11. Réécrivez la boucle de saisie de **triangulation.py** avec une liste et une boucle for, en validant que les rayons soient strictement positifs.

```
villes, rayons = [], []
for i in range(1, 4):
    nom = input(f"Nom de la ville #{i} : ")
    villes.append(nom)
    while True:
        try:
            r = float(input(f"Rayon pour {nom} (km) : "))
            if r <= 0:
                print("Le rayon doit être strictement positif.")
            else:
                rayons.append(r)
                break
        except ValueError:
            print("Veuillez saisir un nombre valide.")
```

PARTIE 3 — QUESTIONS D'ANALYSE DE CODE

Exercice 3.1 — Qualité et robustesse du code

Q12. Dans **triangulation.py**, que se passe-t-il si l'utilisateur entre "bayonne" au lieu de "Bayonne" ? Proposez une correction insensible à la casse.

La comparaison values[0] == ville est strictement case-sensitive. "bayonne" != "Bayonne" donc la ville ne sera pas trouvée. Correction :

```
if values[0].strip().lower() == ville.strip().lower():
```

Q13. Identifiez deux problèmes potentiels liés à l'encodage des fichiers dans les scripts fournis. Comment les scripts gèrent-ils (ou ne gèrent-ils pas) les caractères accentués ?

Problème 1 — triangulation.py utilise encoding='utf-8'. Si le CSV est en latin-1 (ISO-8859-1, fréquent sous Windows), les accents seront mal interprétés.

Problème 2 — villes_vol_oiseau01/02.py utilisent pandas sans préciser l'encoding. Pandas infère automatiquement, ce qui peut échouer sur certains fichiers.

Bonne pratique : utiliser encoding='utf-8-sig' qui gère aussi le BOM (octet de marque d'ordre présent en tête du fichier fourni).

Q14. Que signifie **if __name__ == '__main__'**: dans les scripts ? Quel problème évite-t-elle lors de l'import d'un module ?

__name__ vaut '__main__' uniquement lors de l'exécution directe (python script.py). Lors d'un import (import triangulation), __name__ prend le nom du module. Cette garde empêche que la fonction main() (avec ses input() interactifs) s'exécute automatiquement lors d'un import, ce qui est indésirable dans un contexte de test ou de réutilisation.

Exercice 3.2 — Question de synthèse

Q15. Proposez une architecture en modules Python pour un projet regroupant les fonctionnalités des 4 scripts. Décrivez les modules, leurs fonctions principales et leurs dépendances.

```
geo_utils.py          # Utilitaires géographiques
  - obtenir_coordonnees(ville, fichier) -> tuple | None
  - calculer_distance(coord1, coord2) -> float

carte_utils.py        # Création et manipulation de cartes
  - creer_carte(centre) -> folium.Map
  - ajouter_marqueur(carte, coord, label, couleur)
  - ajouter_cercle(carte, coord, rayon_km, couleur)
  - ajouter_ligne(carte, coord1, coord2)
  - sauvegarder_carte(carte, fichier)

main.py               # Points d'entrée utilisateur
  - distance_villes() # issu de villes_vol_oiseau02.py
  - triangulation()   # issu de triangulation.py
```

Dépendances : main.py ← carte_utils + geo_utils ; carte_utils ← folium ; geo_utils ← csv + math.

BONUS — Pour aller plus loin

Questions bonus (facultatives)

- Implémentez une recherche approximative dans le CSV avec difflib pour suggérer des noms proches si la ville n'est pas trouvée exactement.
- Ajoutez une interface CLI avec argparse pour passer les villes et rayons en arguments plutôt qu'en saisie interactive.
- Calculez et affichez automatiquement l'intersection approximative des 3 cercles en résolvant le système numériquement avec scipy.optimize.
- Exportez les résultats (villes, rayons, distances) dans un fichier JSON en plus de la carte HTML.